

1. Project Title

A Game Start Screen in MS-DOS Using Assembly Language

2. Objective

The purpose of this project is to design and implement an interactive **game start screen** in DOS using Assembly Language. This system allows users to:

- Navigate through menu options using arrow keys
- Enable or disable game sound
- View instructions about the game
- Start a simple game simulation
- Exit the program safely

The project demonstrates the use of **ASCII art**, **keyboard input**, **screen control**, and **PC speaker sound** in Assembly.

3. Tools and Environment

- **Programming Language:** Assembly (8086)
- **IDE / Assembler:** DOSBox
- **Platform:** MS-DOS / DOSBox Emulator
- **Memory Model:** COM program (`[org 0x0100]`)
- **Output Medium:** Text-based screen (VGA text mode)

4. Program Description

This program is a **menu-driven text application** simulating a simple game interface. Main

features include:

- **ASCII Art Title:** Colorful title and subtitle displayed at the top of the menu •

Menu Options: `Start Game`, `Instructions`, `Options`, `Exit` • **Selector**

Arrow: `>>>` indicates the currently selected menu item • **Sound Feature:**

Beep sounds triggered during navigation and selection • **Instructions**

Screen: Explains mission, controls, objectives, and warnings • **Options**

Screen: Allows toggling sound ON or OFF

5. How to Play

- **Arrow Keys (UP / DOWN):** Move the selector
- **ENTER:** Select the highlighted menu option
- **ESC:** Exit the program
- **SPACE (in Options):** Toggle sound ON/OFF

6. Program Flow

Initialization

- Clear the screen (`clrscr`)
- Hide the cursor (`hide_cursor`)
- Draw ASCII art title (`draw_title`) and menu (`draw_menu`)
- Set default values for current selection and sound

Main Loop

- Waits for keyboard input
- Arrow keys update the menu selection
- ENTER executes the current selection
- ESC exits the program

Subroutines

Subroutine Purpose

`clrscr` Clears the screen.

`hide_cursor` Hides the blinking cursor.

`printstr` Prints a string at a given position and color.

`draw_title` Displays the ASCII art title.

`draw_menu` Displays menu options.
Shows selector arrow and
highlights current item.

`update_select` or

`beep` Plays sound via PC speaker (if enabled).

`delay` Implements a timed pause for animations. `start_game`

Displays “Starting game” animation with dots.

`show_instructions`

`toggle_options` Toggles sound ON/OFF in Options menu.
Displays instructions screen.

`exit_program` Displays exit message and terminates the program.

7. Key Code Features

- **Direct video memory manipulation** at `0xB800` for text display and colors
- **Keyboard interrupt** (`INT 16h`) for capturing user input
- Arrow key navigation and ENTER selection logic
- Color highlighting for selected menu items
- Modular subroutines for easy customization

8. Interrupts Used

Interrupt Purpose

`10h` Video services: clear screen, hide cursor, print text.

`16h` Keyboard input (waits for keypress).

1Ah System timer for delays.

21h DOS termination function (4C00h).

9. Notes

- Designed for **80x25 text mode**
- Fully compatible with **DOSBox** or real DOS
- Modular subroutines make it easy to modify text, colors, or menu options
- Sound works only if `sound_enabled = 1`

10. Complete code(with comments)

```
[org 0x0100]
```

```
jmp start  
; Current menu state
```

```
current_selection: db 0
```

```
sound_enabled: db 1
```

; ASCII Art Title

title_line1: db ' _____ ', 0

title_len1: dw 37

title_line2: db ' |||_||| ', 0

title_len2: dw 37

title_line3: db '|--||_| |||- -| ', 0

title_len3: dw 37

title_line4: db '|_|_|_|_|_|_|_|_| ', 0

title_len4: dw 37

subtitle: db '[R A I D E R S]', 0

subtitle_len: dw 17

; Menu options

menu_start: db 'Start Game', 0

menu_start_len: dw 10

menu_instructions: db 'Instructions', 0

menu_inst_len: dw 12

menu_options: db 'Options', 0

menu_opt_len: dw 7

menu_exit: db 'Exit', 0

menu_exit_len: dw 4

; Selector

selector: db '>>>', 0

selector_len: dw 3

; Footer

footer: db 'Use Arrow Keys to Navigate | ENTER to Select | ESC to Exit', 0

footer_len: dw 59

; Messages

msg_starting: db 'Starting game', 0

msg_start_len: dw 13

press_any_key: db 'Press any key to return...', 0

press_key_len: dw 27

exit_msg: db 'Thanks for playing!', 0

exit_msg_len: dw 19

; Instructions screen

inst_title: db 'INSTRUCTIONS', 0

inst_title_len: dw 12

inst_line1: db 'Mission: Defend Earth from alien invaders!', 0

inst_line1_len: dw 43

inst_line2: db 'Controls: Arrow keys to move, SPACE to fire', 0

inst_line2_len: dw 44

inst_line3: db 'Objective: Destroy all enemy ships', 0

inst_line3_len: dw 34

inst_line4: db 'Warning: Avoid enemy fire and collisions!', 0

inst_line4_len: dw 42

; Options screen

opt_title: db 'OPTIONS', 0

opt_title_len: dw 7

opt_sound: db 'Sound: ', 0

opt_sound_len: dw 7

```
opt_on: db 'ON', 0
opt_on_len: dw 2
```

```
opt_off: db 'OFF', 0
```

```
opt_off_len: dw 3
```

```
opt_toggle: db 'Press SPACE to toggle | Any other key to return', 0
```

```
opt_toggle_len: dw 48
```

```
;
=====
```

```
; SUBROUTINES
```

```
;
=====
```

```
; -----
```

```
; clrscr - Clear screen
```

```
; -----
```

```
clrscr:
```

```
    push es
```

```
    push ax
```

```
    push di
```

```
mov ax, 0xb800
```

```
mov es, ax ; point ES to video memory  
mov di, 0 ; start at top-left
```

```
nextloc:
```

```
mov word [es:di], 0x0720 ; clear next char (space + white-on-black)
```

```
add di, 2 ; move to next screen cell
```

```
cmp di, 4000 ; check if entire screen cleared
```

```
jne nextloc ; continue if not
```

```
pop di
```

```
pop ax
```

```
pop es
```

```
ret
```

```
;
```

```
-----  
; hide_cursor - Hide the text cursor
```

```
;
```

hide_cursor:

push ax

push cx

mov ah, 0x01
mov cx, 0x2607

int 0x10

pop cx

pop ax

ret

printstr:

push bp

mov bp, sp

push es

push ax

push cx

push si

push di

mov ax, 0xb800

mov es, ax

mov al, 80

mul byte [bp+10] ; y position add

ax, [bp+12] ; x position shl ax, 1

mov di, ax

mov si, [bp+6] ; string address

mov cx, [bp+4] ; length

mov ah, [bp+8] ; attribute

nextchar:

mov al, [si]

mov [es:di], ax

add di, 2

add si, 1

loop nextchar

pop di

pop si

pop cx

pop ax

pop es

pop bp

ret 10

;

; draw_title - Display ASCII art title

; Uses: printstr subroutine

;

draw_title:

push ax

mov ax, 15

push ax

mov ax, 2

push ax

mov ax, 0x0B ; Cyan

push ax

mov ax, title_line1

push ax

push word [title_len1]

call printstr

mov ax, 10

push ax

mov ax, 3

push ax
mov ax, 0x0B

push ax

mov ax, title_line2

push ax

push word

[title_len2] call

printstr

mov ax, 10

push ax

mov ax, 4

push ax

mov ax, 0x0B

push ax

mov ax, title_line3

push ax

push word

[title_len3] call

printstr

mov ax, 10

push ax
mov ax, 5

push ax

mov ax, 0x0B

push ax

mov ax, title_line4

push ax

push word [title_len4]

call printstr

mov ax, 28

push ax

mov ax, 7

push ax

mov ax, 0x0E ; Yellow push

ax

mov ax, subtitle

push ax

push word [subtitle_len]

call printstr

pop ax

ret

;

; draw_menu - Display menu options

; Uses: printstr subroutine

;

draw_menu:

push ax

mov ax, 32

push ax

mov ax, 11

push ax

mov ax, 0x0F ; White

push ax

mov ax, menu_start

push ax

push word [menu_start_len]

call printstr

mov ax, 32

push ax

mov ax, 13

push ax

mov ax, 0x0F

push ax

mov ax, menu_instructions

push ax

push word [menu_inst_len]

call printstr

mov ax, 32

push ax

mov ax, 15

push ax

mov ax, 0x0F

push ax

mov ax, menu_options

push ax

push word [menu_opt_len]

call printstr

mov ax, 32

push ax

mov ax, 17

push ax

mov ax, 0x0F

push ax

mov ax, menu_exit

push ax

push word [menu_exit_len]

call printstr

mov ax, 20

push ax

mov ax, 22

push ax

mov ax, 0x08 ; Dark gray push

ax

mov ax, footer

push ax

push word [footer_len]

call printstr

pop ax

ret

; ----- ;

update_selector - Show selector next to current menu item ; Uses:

printstr subroutine and direct video memory for highlighting ;

update_selector:

push ax

push bx

push cx

push dx

push es

push di
; Setup video memory access

```
mov ax, 0xb800
```

```
mov es, ax
```

```
; Clear all selectors and reset all menu items to normal color
```

```
mov cx, 4
```

```
mov dl, 11 ; Starting row
```

```
clear_selectors_loop:
```

```
push cx
```

```
push dx
```

```
; Clear selector arrow
```

```
mov ax, 28 ; x position
```

```
push ax
```

```
movzx ax, dl ; y position
```

```
push ax
```

```
mov ax, 0x00 ; Black on black (invisible) push
```

```
ax
```

```
mov ax, selector
```

```
push ax
mov ax, 3
```

```
push ax
```

```
call printstr
```

```
; Reset menu item to normal white color
```

```
pop dx
```

```
movzx ax, dl ; Get row
```

```
mov bl, 80
```

```
mul bl ; Row * 80
```

```
add ax, 32 ; Add column offset for menu text shl ax,
```

```
1 ; Convert to byte offset
```

```
mov di, ax
```

```
; Change attribute of menu item characters to white (0x0F)
```

```
mov cx, 15 ; Max length to update
```

```
mov ah, 0x0F ; White on black
```

```
reset_color_loop:
```

```
mov al, [es:di] ; Read character
```

cmp al, 0 ; Check if end

je reset_done

mov [es:di+1], ah ; Write only attribute byte

add di, 2

loop reset_color_loop

reset_done:

pop cx

add dl, 2

loop clear_selectors_loop

; Draw selector at current position

mov al, [current_selection]

mov ah, 0

mov bl, 2

mul bl

add al, 11

movzx bx, al

; Draw yellow arrow

mov ax, 28

push ax
push bx

mov ax, 0x0E ; Yellow

push ax

mov ax, selector

push ax

push word [selector_len]

call printstr

; Highlight selected menu item with bright yellow/white

movzx ax, bl ; Get row

mov cl, 80

mul cl ; Row * 80

add ax, 32 ; Add column offset

shl ax, 1 ; Byte offset

mov di, ax

; Change attribute to bright white with highlight (0x8F or 0x0E)

mov cx, 15

mov ah, 0x8E ; Bright yellow on black

highlight_loop:
mov al, [es:di]

cmp al, 0

je highlight_done

cmp al, ' '

je skip_space

mov [es:di+1], ah ; Write bright attribute

skip_space:

add di, 2

loop highlight_loop

highlight_done:

pop di

pop es

pop dx

pop cx

pop bx

pop ax

ret

;
; -----
; delay - Create a time delay

; Uses: INT 1Ah (system timer)

;

delay:

push ax

push bx

push cx

push dx

mov ah, 0x00

int 0x1A ; Get tick count in CX:DX mov bx,

dx

add bx, 3 ; Add 3 ticks (~165ms)

delay_loop:

mov ah, 0x00

int 0x1A

cmp dx, bx

jl delay_loop
pop dx

pop cx

pop bx

pop ax

ret

;

; beep - Play a beep sound through PC speaker ;

beep:

push ax

push bx

push cx

; Check if sound is enabled

mov al, [sound_enabled]

cmp al, 0

je beep_skip

; Set up the PIT (Programmable Interval Timer) for sound
mov al, 0xB6 ; Command byte: channel 2, square wave out

out 0x43, al ; Send to command register

; Set frequency (divisor for ~800 Hz)

mov ax, 1193180 / 800 ; 1193180 Hz / desired frequency

out 0x42, al ; Send low byte

mov al, ah

out 0x42, al ; Send high byte

; Turn on speaker

in al, 0x61 ; Read current port 61h value

or al, 0x03 ; Set bits 0 and 1 (speaker on) out

0x61, al

; Short delay for beep duration

mov cx, 1

beep_delay:

push cx

call delay

pop cx
loop beep_delay

; Turn off speaker

in al, 0x61

and al, 0xFC ; Clear bits 0 and 1 (speaker off) out

0x61, al

beep_skip:

pop cx

pop bx

pop ax

ret

;

; start_game - Action for "Start Game" option ; Uses:

INT 1Ah for timing

;

start_game:

call clrscr
mov ax, 25

push ax

mov ax, 12

push ax

mov ax, 0x0A ; Green push

ax

mov ax, msg_starting

push ax

```
push word [msg_start_len]
```

```
call printstr
```

```
; Animate dots
```

```
mov cx, 5
```

```
animate_dots:
```

```
push cx
```

```
call delay
```

```
pop cx
```

```
loop animate_dots
```

```
; Final delay
```

```
mov cx, 3
```

```
final_delay:
```

```
push cx
```

```
call delay
```

```
pop cx
```

```
loop final_delay
```

; Return to menu

call clrscr

call draw_title

call draw_menu

jmp main_loop

;

; show_instructions - Action for "Instructions" option ;

Uses: INT 16h for keyboard input

;

show_instructions:

call clrscr
mov ax, 30

push ax

mov ax, 3

push ax

mov ax, 0x0E ; Yellow push

ax

mov ax, inst_title

push ax

push word [inst_title_len]

call printstr

mov ax, 15

push ax

mov ax, 6

push ax

mov ax, 0x0F

push ax

mov ax, inst_line1

push ax

push word [inst_line1_len]

call printstr

mov ax, 15

push ax

mov ax, 8

push ax

mov ax, 0x0F

push ax

mov ax, inst_line2

push ax

push word

[inst_line2_len] call

printstr

mov ax, 15

push ax

mov ax, 10

push ax

mov ax, 0x0F

push ax

mov ax, inst_line3

push ax

push word

[inst_line3_len] call

printstr

mov ax, 15

push ax

mov ax, 12

push ax

mov ax, 0x0F

push ax

mov ax, inst_line4

push ax

push word

[inst_line4_len] call

printstr

mov ax, 25

push ax

```
mov ax, 20
```

```
push ax
```

```
mov ax, 0x08
```

```
push ax
```

```
mov ax, press_any_key
```

```
push ax
```

```
push word [press_key_len]
```

```
call printstr
```

```
; Wait for keypress
```

```
mov ah, 0x00
```

```
int 0x16
```

```
; Return to menu
```

```
call clrscr
```

```
call draw_title
```

```
call draw_menu
```

```
jmp main_loop
```

;

; toggle_options - Action for "Options" option ; Uses:

INT 16h for keyboard input

;

toggle_options:

call clrscr

mov ax, 33

push ax

mov ax, 3

push ax

mov ax, 0x0E

push ax

mov ax, opt_title

push ax

push word [opt_title_len]

call printstr

mov ax, 25

push ax

mov ax, 8

push ax

mov ax, 0x0F

push ax
mov ax, opt_sound

push ax

push word [opt_sound_len]

call printstr

; Display sound state

mov al, [sound_enabled]

cmp al, 1

je sound_on_display

mov ax, 32

push ax

mov ax, 8

push ax

mov ax, 0x0C ; Red push

ax

mov ax, opt_off

push ax

push word [opt_off_len]

call printstr

jmp display_toggle

sound_on_display:

mov ax, 32

push ax

mov ax, 8

push ax

mov ax, 0x0A ; Green push

ax

mov ax, opt_on

push ax

```
push word [opt_on_len]
```

```
call printstr
```

```
display_toggle:
```

```
mov ax, 20
```

```
push ax
```

```
mov ax, 11
```

```
push ax
```

```
mov ax, 0x07
```

```
push ax
```

```
mov ax, opt_toggle
```

```
push ax
```

```
push word [opt_toggle_len]
```

```
call printstr
```

```
; Wait for keypress
```

```
mov ah, 0x00
```

```
int 0x16
```

```
; Check if SPACE was pressed
```

```
cmp al, ''
```

```
jne options_exit
```

```
; Toggle sound setting
```

```
mov al, [sound_enabled]
```

```
xor al, 1
```

```
mov [sound_enabled],
```

```
al jmp toggle_options
```

```
options_exit:
```

```
call clrscr
```

```
call draw_title
```

```
call draw_menu
```

```
jmp main_loop
```

```
;
```

```
=====
```

```
; MAIN PROGRAM
```

;

=====

start:

; Initialize

call clrscr

call hide_cursor

call draw_title

call draw_menu

mov byte [current_selection], 0

mov byte [sound_enabled], 1
main_loop:

; Update selector position

call update_selector

; Wait for keypress (INT 16h)

mov ah, 0x00

int 0x16

; Check for extended keys (arrow keys)

cmp al, 0

jne check_enter

cmp ah, 0x48 ; UP arrow je

move_up

cmp ah, 0x50 ; DOWN arrow je

move_down

jmp main_loop

check_enter:

cmp al, 0x0D ; ENTER key

je execute_selection

cmp al, 0x1B ; ESC key

je exit_program

jmp main_loop

move_up:

call beep ; Play sound on movement mov al,

[current_selection]

cmp al, 0

je wrap_to_bottom

dec byte [current_selection]

jmp main_loop

wrap_to_bottom:

mov byte [current_selection], 3

jmp main_loop

move_down:

call beep ; Play sound on movement
mov al, [current_selection]

cmp al, 3

je wrap_to_top

inc byte [current_selection]

jmp main_loop

wrap_to_top:

mov byte [current_selection], 0

jmp main_loop

execute_selection:

call beep ; Play sound on selection call beep

; Double beep for selection mov al,

[current_selection]

cmp al, 0

je start_game

cmp al, 1

je show_instructions

cmp al, 2

je toggle_options

cmp al, 3

je exit_program

jmp main_loop

```
;
```

```
; exit_program - Graceful exit to DOS
```

```
;
```

```
exit_program:
```

```
    ; Restore cursor
```

```
    mov ah, 0x01
```

```
    mov cx, 0x0607
```

```
    int 0x10
```

```
    call clrscr
```

```
    mov ax, 28
```

```
    push ax
```

```
    mov ax, 12
```

```
    push ax
```

```
    mov ax, 0x0E
```

```
push ax

mov ax, exit_msg

push ax

push word [exit_msg_len]

call printstr


call delay


; Terminate program

mov ax, 0x4C00

int 0x21
```

Conclusion

This project demonstrates the power of **Assembly Language for low-level DOS programming**. Key learning outcomes include:

- Handling keyboard input

- Designing text-based user interfaces
- Using a PC speaker for sound effects
- Manipulating screen content via direct memory access
- Implementing structured programming with subroutines

The final output is a fully functional, interactive **game menu system**, which can be extended for actual games or simulations.